Computer Vision

Project Report

Natural Scene Image Classification

Using Convolutional Neural Networks

PyTorch, TensorFlow/Keras, Flask

Student : Tikome Nana Ismaila

Computer Security

Project Overview and Dataset

Context

This project aims to build a complete image classification pipeline applied to the *Intel Image Classification Dataset*. Two CNN models are trained independently — one using **PyTorch**, the other using **TensorFlow/Keras** — and then deployed via a Flask web application allowing a user to submit an image and obtain a real-time prediction.

Dataset Description

The dataset contains approximately **17,000** color images of natural scenes, organized into six categories: *buildings, forest, glacier, mountain, sea, street.*

Class	Training	Test	Total
buildings	2,191	437	2,628
forest	2,271	474	2,745
glacier	2,404	553	2,957
mountain	2,512	525	3,037
sea	2,274	510	2,784
street	2,382	501	2,883
Total	**14,034**	**3,000**	**17,034**

The training set is split into 80% training / 20% validation (≈11,200 / 2,800 images).

Preprocessing and Augmentation

All images are resized to **150×150 pixels**. Pixels are normalized (ImageNet means and standard deviations for PyTorch; division by 255 for Keras). During training: random horizontal flip, ±15°rotation, brightness and contrast variations (±20%). During testing, only resizing and normalization are applied.

CNN Model Architectures

Model 1 — PyTorch

The architecture consists of three convolutional blocks. Each block includes two 3×3 convolutions (same padding), Batch Normalization, ReLU activation, and 2×2 Max Pooling. The channel progression is $3 \to 32 \to 64 \to 128$; spatial resolution decreases from 150 to 75, 37, then 18 pixels.

A Flatten layer produces a 41,472-dimensional vector, followed by two dense layers (512 then 6 outputs) with Dropout. The **Adam** optimizer (lr = 0.001) minimizes Cross-Entropy Loss.

Model 2 — TensorFlow/Keras

The three-block structure is identical, but Flatten is replaced by Global Average Pooling (GAP). GAP produces a 128-dimensional vector, drastically reducing the number of parameters (≈900K vs ≈21M), which helps limit overfitting and speeds up convergence.

Final classifier: Dense(256, ReLU) → Dropout(0.5) → Dense(6, softmax).

Criterion	PyTorch CNN	Keras CNN
Final pooling	Flatten (41,472 neurons)	Global Avg Pooling (128)
Number of parameters	≈ 21 million	≈ 900 thousand
Save format	`.pth`	`.keras`

Training and Results

Configuration

Both models are trained for 15 epochs with a batch size of 32. EarlyStopping (patience = 5) stops training if validation accuracy stagnates. The learning rate is automatically halved (ReduceLROnPlateau, patience = 3). GPU acceleration is supported natively in both frameworks.

Expected Performance

Metric	PyTorch	Keras
Test accuracy	85–90%	85–91%
GPU training time	≈5–8 min	≈5–8 min
CPU training time	≈60–90 min	≈45–75 min

The Keras model converges more smoothly and is less prone to overfitting while achieving comparable performance.

Web Application and Deployment

Flask Backend

The application (`app.py`) exposes two routes:

- `GET /` — serves the HTML interface.
- `POST /predict` — processes an image and returns prediction results.

Frontend and Deployment

The interface is built with HTML/CSS/JavaScript. The application can be deployed on PythonAnywhere or Fly.io.

Challenges and Conclusion

Challenges

- Framework differences (PyTorch vs TensorFlow formats).
- Overfitting mitigation.
- Model loading in PyTorch.
- Deployment without GPU.

Conclusion

This project demonstrates a complete computer vision pipeline from data preparation to deployment. Both models achieve 85–91% accuracy. Future work includes transfer learning and Grad-CAM visualization.